



# Week 5 Workshop

# Indexing and Query

# Processing

**IFN647**

Tutor

School of Computer Science  
Queensland University of Technology

# Workshop Objectives

- Learn how to process a collection of documents in a folder using the **glob** module.
- Develop a Python function to construct **an inverted index** for a given set of documents.
- Understand how to perform **query processing** in Python.
- *Learn how to manage **file paths for data access** in Python (Python skills).*
  - *Assume the Jupyter Notebook is executed from the working directory.*
  - *Use functions from the os module to identify the current working directory (current\_path) and construct the path to the data directory (data\_path).*
  - *Note that after calling index\_docs, the working directory may be modified; therefore, it is important to reset it back to the original working directory to ensure consistent file access.*

## Tasks

- Task1 practice (15min) / solution (10min)
- Task2&3 practice (30min) / solution (10min)
- Task4 practice (10min) / solution (5min)

# Workshop Task 1

**Task 1** For the given XML documents (in [w5\\_data-1.zip](#)), design a python function *index\_docs()* to index them (please remove stop words and index stems only). The returned index should be a dictionary {term:{docID1:freq1, DocID2:freq2}, ...}

The following are a sample of outcomes, where '6146' and '741299' are the document id, respectively.

```
Index = {'argentin': {'6146': 1}, 'bond': {'6146': 1}, 'slight': {'6146': 2}, 'higher':  
{ '6146': 1}, 'small': {'6146': 1}, 'technic': {'6146': 2}, 'bounc': {'6146': 2},  
'wednesday': {'6146': 1}, ..., 'time': {'741299': 1}, 'abl': {'741299': 1}, 'push':  
{ '741299': 1}, 'hard': {'741299': 1}, 'big': {'741299': 1}, 'third': {'741299': 1},  
'finish': {'741299': 1}, 'porsch': {'741299': 1}, 'franc': {'741299': 1}, 'bob':  
{ '741299': 1}, 'wollek': {'741299': 1}, 'yannick': {'741299': 1}, 'dalma': {'741299':  
1}, 'belgian': {'741299': 2}, 'thierri': {'741299': 1}, 'boutsen': {'741299': 1},  
'former': {'741299': 1}, 'formula': {'741299': 1}, 'one': {'741299': 1}, 'driver':  
{ '741299': 1}, 'switch': {'741299': 1}, 'normal': {'741299': 1}, 'share': {'741299':  
1}, 'han': {'741299': 1}, 'stuck': {'741299': 1}, 'follow': {'741299': 1}, 'power':  
{ '741299': 1}, 'steer': {'741299': 1}, 'failur': {'741299': 1}}
```

# From previous week...

The return value of function `parse_doc`

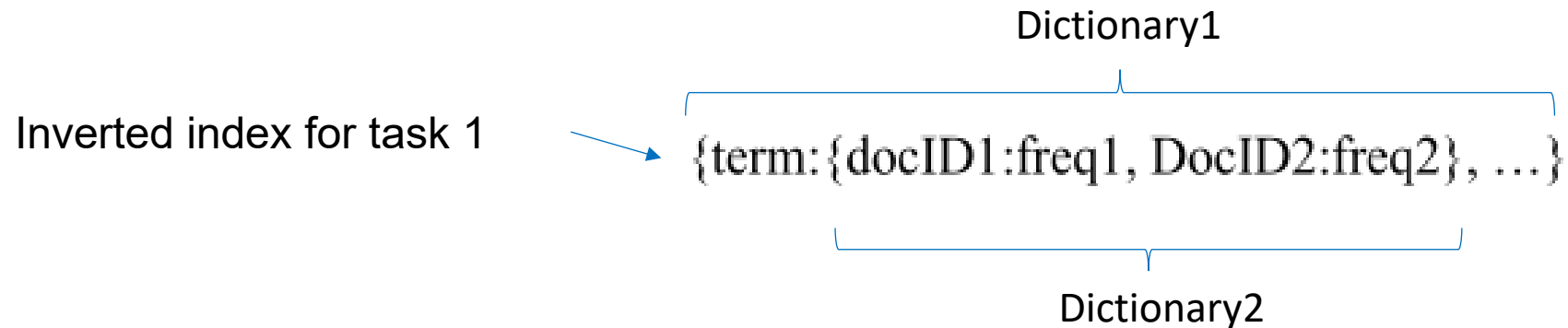
```
(133, {'6146': {'argentine': 1, 'bonds': 1, 'slightly': 1, 'higher': 1, 'small': 1, 'technical': 2, 'bounce': 2, 'wednesday': 1, 'amid': 1, 'low': 1, 'volume': 1, 'trader': 2, 'large': 1, 'foreign': 1, 'bank': 1, 'slight': 1, 'opening': 1, 'expect': 1, 'prices': 1, 'change': 1, 'much': 1, 'during': 1, 'session': 1, 'market': 2, 'moving': 1, 'news': 1, 'expected': 2, 'percent': 1, 'dollar': 1, 'denominated': 1, 'bocon': 1, 'previsional': 1, 'due': 2, 'rose': 2, 'argentina': 2, 'frb': 1, 'quot': 2, 'general': 1, 'uncertainty': 1, 'pointing': 1, 'events': 1, 'waiting': 1, 'including': 1, 'passage': 1, 'government': 1, 'new': 1, 'economic': 1, 'measures': 1, 'through': 1, 'congress': 1, 'now': 1, 'until': 1, 'early': 1, 'october': 1, 'addition': 1, 'traders': 1, 'awaiting': 1, 'meeting': 1, 'friday': 1, 'between': 1, 'economy': 1, 'minister': 1, 'roque': 1, 'fernandez': 1, 'international': 1, 'monetary': 1, 'fund': 1, 'delegation': 1, 'fiscal': 1, 'deficit': 1, 'axel': 1, 'bugge': 1, 'buenos': 1, 'aires': 1, 'newsroom': 1}}})
```

## Expected Output...

```
Index = {'argentin': {'6146': 1}, 'bond': {'6146': 1}, 'slight': {'6146': 2}, 'higher': {'6146': 1}, 'small': {'6146': 1}, 'technic': {'6146': 2}, 'bounc': {'6146': 2}, 'wednesday': {'6146': 1}, ..., 'time': {'741299': 1}, 'abl': {'741299': 1}, 'push': {'741299': 1}, 'hard': {'741299': 1}, 'big': {'741299': 1}, 'third': {'741299': 1}, 'finish': {'741299': 1}, 'porsch': {'741299': 1}, 'franc': {'741299': 1}, 'bob': {'741299': 1}, 'wollek': {'741299': 1}, 'yannick': {'741299': 1}, 'dalma': {'741299': 1}, 'belgian': {'741299': 2}, 'thierri': {'741299': 1}, 'boutsen': {'741299': 1}, 'former': {'741299': 1}, 'formula': {'741299': 1}, 'one': {'741299': 1}, 'driver': {'741299': 1}, 'switch': {'741299': 1}, 'normal': {'741299': 1}, 'share': {'741299': 1}, 'han': {'741299': 1}, 'stuck': {'741299': 1}, 'follow': {'741299': 1}, 'power': {'741299': 1}, 'steer': {'741299': 1}, 'failur': {'741299': 1}}
```

# Workshop Task 1 Knowledge Hint – Inverted Index

- Indexes are **data structures** designed to make search faster.
- In this task, you're building an **inverted index**, which is a structure used in search engines like Google.
- It maps terms to the documents they occur in. Each term points to a list of document identifiers where that term appears, along with additional information such as term frequency, position, etc.



# Workshop Task 1 Knowledge Hint – glob

The **glob** module in Python

The **glob** module in Python is used for file path expansion.  
It helps find files and directories whose names match a specified pattern.

Example:

```
import glob  
glob.glob("*.xml")
```

is used to find all files in the current directory with an extension of ".xml".

- **glob.glob()** is a function from the **glob** module.
- **"\*.xml"** is the pattern provided to **glob.glob()**. It specifies that we're looking for files with names ending with ".xml".
- **glob.glob("\*.xml")** returns a list of file paths matching the specified pattern.

# Workshop Task 1 - Solution

**Task 1** For the given XML documents (in [w5\\_data-1.zip](#)), design a python function *index\_docs()* to index them (please remove stop words and index stems only). The returned index should be a dictionary {term:{docID1:freq1, DocID2:freq2}, ...}

The following are a sample of outcomes, where '6146' and '741299' are the document id, respectively.

```
Index = {'argentin': {'6146': 1}, 'bond': {'6146': 1}, 'slight': {'6146': 2}, 'higher':  
{ '6146': 1}, 'small': {'6146': 1}, 'technic': {'6146': 2}, 'bounc': {'6146': 2},  
'wednesday': {'6146': 1}, ..., 'time': {'741299': 1}, 'abl': {'741299': 1}, 'push':  
{ '741299': 1}, 'hard': {'741299': 1}, 'big': {'741299': 1}, 'third': {'741299': 1},  
'finish': {'741299': 1}, 'porsch': {'741299': 1}, 'franc': {'741299': 1}, 'bob':  
{ '741299': 1}, 'wollek': {'741299': 1}, 'yannick': {'741299': 1}, 'dalma': {'741299':  
1}, 'belgian': {'741299': 2}, 'thierri': {'741299': 1}, 'boutsen': {'741299': 1},  
'former': {'741299': 1}, 'formula': {'741299': 1}, 'one': {'741299': 1}, 'driver':  
{ '741299': 1}, 'switch': {'741299': 1}, 'normal': {'741299': 1}, 'share': {'741299':  
1}, 'han': {'741299': 1}, 'stuck': {'741299': 1}, 'follow': {'741299': 1}, 'power':  
{ '741299': 1}, 'steer': {'741299': 1}, 'failur': {'741299': 1}}
```

## Wk2\_Solution (Task 2) – Explained @ Worshop1

```
import glob, os
import string

def parse_doc(inputpath, stop_ws):
    coll = {}
    os.chdir(inputpath)
    # Example of "inputpath" when you Run...Customized in Mac:
    # /Users/li/IFN647/wk2_workshop_solutions

    myfile=open('6146.xml')

    curr_doc = {}
    start_end = False

    file_=myfile.readlines()
    word_count = 0 #wk2
    for line in file_:
        line = line.strip()
        #print(line)
        if(start_end == False):
            if line.startswith("<newsitem "):
                for part in line.split():
                    if part.startswith("itemid="):
                        docid = part.split("=")[1].split("\\"")[1]
                        break
            if line.startswith("<text>"):
                start_end = True
        elif line.startswith("</text>"):
            break
        else:
            line = line.replace("<p>", "").replace("</p>", "")
            line = line.translate(str.maketrans('', '', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
            line = line.replace("\\s+", " ")
            for term in line.split():
                word_count += 1 #wk2
                term = term.lower() #wk2
                if len(term) > 2 and term not in stop_words: #wk2
                    try:
                        curr_doc[term] += 1
                    except KeyError:
                        curr_doc[term] = 1

    myfile.close()
    return(word_count, {docid:curr_doc})
# return a tuple, the first element is the number of words in <text> and
# the second one is a dirctionary that includes only one pair of doc_id and a disctionary of term_frequency pairs
```



# Wk2\_Solution (Task 2) – Explained @ Worshop1

```
import glob, os
import string
from stemming.porter2 import stem

def parse_doc(inputpath, stop_ws):
    os.chdir(inputpath)
    # Example of "inputpath" when you Run...Customized in Mac:
    # /Users/li/IFN647/wk2_workshop_solutions

    myfile=open('6146.xml')

    curr_doc = {}
    start_end = False

    file_=myfile.readlines()
    word_count = 0
    for line in file_:
        line = line.strip()
        #print(line)
        if(start_end == False):
            if line.startswith("<newsitem "):
                from stemming.porter2 import stem
                for part in line.split():
                    if part.startswith("itemid="):
                        docid = part.split("=")[1].split("\\")[1]
                        break
            if line.startswith("<text>"):
                start_end = True
        elif line.startswith("</text>"):
            break
        else:
            line = line.replace("<p>", "").replace("</p>", "")
            line = line.translate(str.maketrans('', '', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
            line = line.replace("\\s+", " ")
            for term in line.split():
                word_count += 1
                term = term.lower()
                if len(term) > 2 and term not in stop_ws:
                    try:
                        curr_doc[term] += 1
                    except KeyError:
                        curr_doc[term] = 1

    myfile.close()
    return(word_count, {docid:curr_doc})
# return a tuple, the first element is the number of words in <text> and
# the second one is a dirctionary that includes only one pair of doc_id and a disctionary of term_frequency pairs
```

Change the function name to index\_docs

Change as to read all .xml files

Initialize Index { }

Change to create Index data structure

`glob.glob("*.xml")` returns a list of filenames (strings) that match the pattern `*.xml`.

```
import glob
for name in glob.glob('*'):
    print(name)
```

```
6146.xml
741299newsML.xml
common-english-words.txt
example.html
seq1.txt
seq3.txt
Task4.ipynb
Test.ipynb
Test5.ipynb
test_wk2.ipynb
week02_solution.ipynb
wk3_solutions.ipynb
wk4_solutions.ipynb
wk5_solutions.ipynb
workshop_w2_test.ipynb
```

```
import glob
for name in glob.glob('*.xml'):
    print(name)
```

```
6146.xml
741299newsML.xml
```

# Workshop Task 1-Solution...

```
import glob, os
import string
from stemming.porter2 import stem
```

```
def index_docs(inputpath, stop_words):
```

```
    Index = {} # initialize the index
```

```
    os.chdir(inputpath)
```

```
    for file_ in glob.glob("*.xml"):
```

```
        start_end = False
```

```
        for line in open(file_):
```

```
            line = line.strip()
```

```
            if(start_end == False):
```

```
                if line.startswith("<newsitem "):
```

```
                    for part in line.split():
```

```
                        if part.startswith("itemid="):
```

```
                            docid = part.split("=")[1].split("\\"")[1]
```

```
                            break
```

```
                if line.startswith("<text>"):
```

```
                    start_end = True
```

```
            elif line.startswith("</text>"):
```

```
                break
```

```
        else:
```

```
            line = line.replace("<p>", "").replace("</p>", "")
```

```
            line = line.translate(str.maketrans('', '', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
```

```
            for term in line.split():
```

```
                term = stem(term.lower())
```

```
                if len(term) > 2 and term not in stop_words:
```

```
                    try:
```

```
                        try:
```

```
                            Index[term][docid] += 1
```

```
                        except KeyError:
```

```
                            Index[term][docid]=1
```

```
                    except KeyError:
```

```
                        Index[term] = {docid:1}
```

```
    return Index
```

← This is our inverted index dictionary

← Loop through all .xml files

← Reads one line of each file at a time

← {term:{docID1:freq1, DocID2:freq2}, ...}

If term & docid already exist → increment frequency

If term exists but docid is new → add docid

If term is new → create new entry in the index

# Workshop Task 1 - Solution Execution (Jupyter notebook / Cmd)...

Run through Jupyter note book... (Refer Jupyter notebook)

# Workshop Task 2 & 3

**Task 2.** Design a python function *doc\_at\_a\_time(I, Q)*, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

**Input:**

I - Inverted Index - {term: {docID1:freq1, DocID2:freq2}, ...}

Q – {'term1': freq1, 'term2': freq2, ...} where term1, term2, ... terms in a query.

**Output:**

{docID1: relevanceScore, docID2: relevanceScore }

**Task 3.** Design a python function *term\_at\_a\_time(I, Q)*, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

# Workshop Task 2&3 Hint

**Relevance Score of a document (D) to a given query (Q)**

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

$f_i$  is a document feature function  
 $g_i$  is a query feature function

# Workshop Task 2&3 Hint

Example: Query {salt:1, water:1, tropical:1}

## Document-at-a-time

- Calculates complete scores for documents by processing all term lists, one document at a time.

|          |     |     |     |     |
|----------|-----|-----|-----|-----|
| salt     | 1:1 |     |     | 4:1 |
| water    | 1:1 | 2:1 |     | 4:1 |
| tropical | 1:2 | 2:2 | 3:1 |     |
| score    | 1:4 | 2:3 | 3:1 | 4:2 |

## Term-at-a-time

- Accumulates scores for documents by processing term lists one at a time.

|                    |     |     |     |     |
|--------------------|-----|-----|-----|-----|
| salt               | 1:1 | 4:1 |     |     |
| partial scores     | 1:1 | 4:1 |     |     |
|                    |     |     |     |     |
| old partial scores | 1:1 |     | 4:1 |     |
| water              | 1:1 | 2:1 | 4:1 |     |
| new partial scores | 1:2 | 2:1 | 4:2 |     |
|                    |     |     |     |     |
| old partial scores | 1:2 | 2:1 | 4:2 |     |
| tropical           | 1:2 | 2:2 | 3:1 |     |
| final scores       | 1:4 | 2:3 | 3:1 | 4:2 |

# Workshop Task 2&3 Hint

Example: Query {salt:1, water:1, tropical:1}

## Document-at-a-time

- Calculates complete scores for documents by processing all term lists, one document at a time.

| Doc ID   | Term Freq in Doc |     |     |     |
|----------|------------------|-----|-----|-----|
| salt     | 1:1              |     |     | 4:1 |
| water    | 1:1              | 2:1 |     | 4:1 |
| tropical | 1:2              | 2:2 | 3:1 |     |
| score    | 1:4              | 2:3 | 3:1 | 4:2 |

$$1 * 1 + 1 * 1 + 2 * 1 = 4$$

$$1 * 1 + 2 * 1 = 3$$

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

$f_i$  is a document feature function

$g_i$  is a query feature function



# Workshop Task 2&3 Hint

Example: Query {salt:1, water:1, tropical:1}

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

$f_i$  is a document feature function  
 $g_i$  is a query feature function

## Term-at-a-time

- Accumulates scores for documents by processing term lists one at a time.

| Doc 1       | Doc 2       | Doc 3       | Doc 4       |
|-------------|-------------|-------------|-------------|
| $1 * 1 = 1$ |             |             | $1 * 1 = 1$ |
| +           |             |             | +           |
| $1 * 1 = 1$ | $1 * 1 = 1$ |             | $1 * 1 = 1$ |
| 2           | 1           |             | 2           |
| +           |             | +           |             |
| $2 * 1 = 2$ | $2 * 1 = 2$ | $1 * 1 = 1$ |             |
| 4           | 3           | 1           | 2           |

Doc ID

Term Freq in Doc

salt

1:1

4:1

partial scores

1:1

4:1

old partial scores

1:1

4:1

water

1:1

2:1

4:1

new partial scores

1:2

2:1

4:2

old partial scores

1:2

2:1

4:2

tropical

1:2

2:2

3:1

final scores

1:4

2:3

3:1

4:2

# Workshop Task 2&3 Hint

**Task 2.** Design a python function *doc\_at\_a\_time(I, Q)*, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

**Task 3.** Design a python function *term\_at\_a\_time(I, Q)*, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query Q (a term:freq dictionary).

## Input:

I - Inverted Index

{'argentin': {'6146': 1}, 'bond': {'6146': 1}, ... 'lehto': {'741299': 2}, 'finland': {'741299': 1}, ... 'formula': {'741299': 1},... 'one': {'741299': 1},... }

Q – Query → {'formula': 1, 'one': 1}

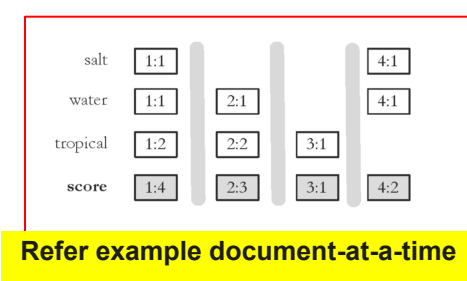
Output: {'6146': 0, '741299': 2}

# Workshop Task 2 - Solution using doc-at-a-time

**Task 2.** Design a python function *doc\_at\_a\_time(I, Q)*, where index I is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query *Q* (a term:freq dictionary).

**Input:** I - Inverted Index  
{'argentin': {'6146': 1}, 'bond': {'6146': 1}, ... 'lehto': {'741299': 2}, 'finland': {'741299': 1},..., 'formula': {'741299': 1},... 'one': {'741299': 1},... }  
Q – Query {'formula': 1, 'one': 1}

**Output:** {'6146': 0, '741299': 2}



# Workshop Task 2 - Solution using doc-at-a-time

**Task 2.** Design a python function `doc_at_a_time(I, Q)`, where index `I` is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query `Q` (a term:freq dictionary).

Refer example document-at-a-time

|          |     |     |     |     |
|----------|-----|-----|-----|-----|
| salt     | 1:1 |     |     | 4:1 |
| water    | 1:1 | 2:1 |     | 4:1 |
| tropical | 1:2 | 2:2 | 3:1 |     |
| score    | 1:4 | 2:3 | 3:1 | 4:2 |

**Input:** I - Inverted Index

{'argentin': {'6146': 1}, 'bond': {'6146': 1}, ... 'lehto': {'741299': 2}, 'finland': {'741299': 1},..., 'formula': {'741299': 1},... 'one': {'741299': 1, '809495': 1},... }

Q – Query {'formula': 1, 'one': 1}

**Output:** {'6146': 0, '741299': 2}

```
1 def doc_at_a_time(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
2     L={} # L is the selected inverted lists
3     R={} # R is a dictionary of docId:relevance
4     for list in I.items():
5         for id in list[1].items(): # get all document IDs with value 0
6             R[id[0]]=0 → R: {'6146': 0, '809495': 0, '741299': 0, '809481': 0}
7             if (list[0] in Q): # select inverted lists based on the query
8                 L[list[0]]= I[list[0]] → {'formula': {'741299': 1}, 'one': {'741299': 1, '809495': 1}}
```

# Workshop Task 2 - Solution using doc-at-a-time

**Task 2.** Design a python function `doc_at_a_time(I, Q)`, where index `I` is a Dictionary of term:Dictionary of (itemId:freq), which returns a dictionary of docId:relevance for the given query `Q` (a term:freq dictionary).

|          |     |     |     |     |
|----------|-----|-----|-----|-----|
| salt     | 1:1 |     |     | 4:1 |
| water    | 1:1 | 2:1 |     | 4:1 |
| tropical | 1:2 | 2:2 | 3:1 |     |
| score    | 1:4 | 2:3 | 3:1 | 4:2 |

Refer example document-at-a-time

Input: I - Inverted Index

{'argentin': {'6146': 1}, 'bond': {'6146': 1}, ... 'lehto': {'741299': 2}, 'finland': {'741299': 1},..., 'formula': {'741299': 1},... 'one': {'741299': 1, '809495': 1},... }

Q – Query {'formula': 1, 'one': 1}

Output: {'6146': 0, '741299': 2}

```
1 def doc_at_a_time(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
2     L={} # L is the selected inverted lists
3     R={} # R is a dictionary of docId:relevance
4     for list in I.items():
5         for id in list[1].items(): # get all document IDs with value 0
6             R[id[0]]=0
7             if (list[0] in Q): # select inverted lists based on the query
8                 L[list[0]]= I[list[0]]
9     for (d, sd) in R.items():
10         for (term, f) in L.items():
11             if (d in f):
12                 sd = sd + f[d]*Q[term]
13         R[d] = sd
14     return R
```

Annotations in the code:

- Line 6: `R[id[0]]=0` is annotated with a red box and an arrow pointing to the initial state of `R`: `R: {'6146': 0, '809495': 0, '741299': 0, '809481': 0}`.
- Line 8: `L[list[0]]= I[list[0]]` is annotated with a red box and an arrow pointing to the selected inverted lists: `{'formula': {'741299': 1}, 'one': {'741299': 1, '809495': 1}}`.
- Line 12: `sd = sd + f[d]*Q[term]` is annotated with a red box and an arrow pointing to the calculation of the relevance score for the 'one' term.

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

$f_i$  is a document feature function  
 $g_i$  is a query feature function

$Q = \{\text{'formula':1, 'one':1}\}$

$L = \{\text{'formula':}\{741299:1\}, \text{'one':}\{741299:1, 809495:1\}\}$

|         |                |                |
|---------|----------------|----------------|
| one     | 741299 : 1 * 1 | 809495 : 1 * 1 |
| formula | 741299 : 1 * 1 |                |
| Score   | 741299 : 2     | 809495 : 1     |

## Task2 sample outputs:

Document\_at\_a\_time result-----

Document ID: 741299 and relevance weight: 2

Document ID: 809495 and relevance weight: 1

# Workshop Task 3 – Solution using term-at-a-time

I - Inverted Index

{'argentin': {'6146': 1}, 'bond': {'6146': 1}, ... 'lehto': {'741299': 2}, 'finland': {'741299': 1},..., 'formula': {'741299': 1},... 'one': {'741299': 1, '809495': 1}, ... }

Q – Query {'formula': 1, 'one': 1}

{'6146': 0, '741299': 2}

```
1 def term_at_a_time(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
2     L={} # L is the selected inverted lists
3     R={} # R is a dictionary of docId:relevance
4     for list in I.items():
5         for id in list[1].items(): # get all document IDs with value 0
6             R[id[0]]+=0
7             if (list[0] in Q): # select inverted lists based on the query
8                 L[list[0]]+= I[list[0]]
```

→ R: {'6146': 0, '809495': 0, '741299': 0, '809481': 0}

→ {'formula': {'741299': 1}, 'one': {'741299': 1, '809495': 1}}

|                    |     |     |     |     |
|--------------------|-----|-----|-----|-----|
| salt               | 1:1 | 4:1 |     |     |
| partial scores     | 1:1 | 4:1 |     |     |
|                    |     |     |     |     |
| old partial scores | 1:1 | 4:1 |     |     |
| water              | 1:1 | 2:1 | 4:1 |     |
| new partial scores | 1:2 | 2:1 | 4:2 |     |
|                    |     |     |     |     |
| old partial scores | 1:2 | 2:1 | 4:2 |     |
| tropical           | 1:2 | 2:2 | 3:1 |     |
| final scores       | 1:4 | 2:3 | 3:1 | 4:2 |

Refer example term-at-a-time

# Workshop Task 3 – Solution using term-at-a-time

**Input:** I - Inverted Index  
 {'argentin': {'6146': 1}, 'bond': {'6146': 1}, ... 'lehto': {'741299': 2}, 'finland': {'741299': 1},..., 'formula': {'741299': 1},... 'one': {'741299': 1, '809495': 1},... }  
**Output:** {'6146': 0, '741299': 2}

```

1 def term_at_a_time(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
2     L={} # L is the selected inverted lists
3     R={} # R is a dictionary of docId:relevance
4     for list in I.items():
5         for id in list[1].items(): # get all document IDs with value 0
6             R[id[0]] = 0
7             if (list[0] in Q): # select inverted lists based on the query
8                 L[list[0]] = I[list[0]]
9             for (term, li) in L.items(): # traversal of the selected inverted list
10                for (d, f) in li.items(): # for each occurrence of doc, update R
11                    R[d] = R[d] + f*Q[term]
12    return R
    
```

## Task3 sample outputs:

Term\_at\_a\_time result -----

Document ID: 741299 and relevance weight: 2

Document ID: 809495 and relevance weight: 1

$$R(Q, D) = \sum_i g_i(Q) f_i(D)$$

$f_i$  is a document feature function  
 $g_i$  is a query feature function

Q = {'formula':1, 'one':1}      L = {'formula':{'741299':1}, 'one':{'741299':1, '809495':1}}

|                |                |                |
|----------------|----------------|----------------|
| formula        | 741299 : 1 * 1 |                |
| Partial scores | 741299 : 1     |                |
| one            | 741299 : 1 * 1 | 809495 : 1 * 1 |
| Final scores   | 741299 : 2     | 809495 : 1     |

|                    |     |     |         |
|--------------------|-----|-----|---------|
| salt               | 1:1 | 4:1 |         |
| partial scores     | 1:1 | 4:1 |         |
| old partial scores | 1:1 |     | 4:1     |
| water              | 1:1 | 2:1 | 4:1     |
| new partial scores | 1:2 | 2:1 | 4:2     |
| old partial scores | 1:2 | 2:1 | 4:2     |
| tropical           | 1:2 | 2:2 | 3:1     |
| final scores       | 1:4 | 2:3 | 3:1 4:2 |

Refer example term-at-a-time



# Workshop Task 4

**Task 4.** Design a python main program to call the above three functions for a query, e.g.,  
Query = {'formula':1, 'one':1}.

## **The sample outputs:**

Document\_at\_a\_time result-----

Document ID: 741299 and relevance weight: 2

...

Term\_at\_a\_time result -----

Document ID: 741299 and relevance weight: 2

...

## **Basic idea:**

1. Obtain the stopping word list
2. Obtain Inverted Index :index\_docs (data\_path, stop\_words)
3. Call doc\_at\_a\_time(Index, Query)
4. Call term\_at\_a\_time(Index, Query)
5. Ranking the result: sorted function
6. Retrieval each result and print the output



```

1 import sys
2
3 curr_path=os.getcwd()
4 print(curr_path)
5
6 stopwords_f = open('common-english-words.txt', 'r')
7 stop_words = stopwords_f.read().split(',')
8 stopwords_f.close()
9
10 data_path = curr_path+'/data'
11 Index = index_docs(data_path, stop_words)
12
13 os.chdir(curr_path)
14 print(Index)
15
16 Query = {'formula':1, 'one':1}
17 result1 = doc_at_a_time(Index, Query)
18 result2 = term_at_a_time(Index, Query)
19 x1 = sorted(result1.items(), key=lambda x: x[1],reverse=True)
20 x2 = sorted(result2.items(), key=lambda x: x[1],reverse=True)
21 print('Document_at_a_time result-----')
22 for (id, w) in x1:
23     if w>0:
24         print('Document ID: ' + id + ' and relevance weight: ' + str(w))
25 print('Term_at_a_time result -----')
26 for (id, w) in x2:
27     if w>0:
28         print('Document ID: ' + id + ' and relevance weight: ' + str(w))

```

Generate the stopping word list

Obtain the path of the data collection  
Call the index\_docs function and get the returned inverted index  
*#create an index for all terms in <text>,  
#data structure {'w1':{'ID1':2, 'ID2':1}, 'w2':{'ID3':1, 'ID1':3}}*

Change to the current working directory

Call the doc\_at\_a\_time and term\_at\_a\_time functions with the inverted index and the given query

Sort the returned dictionary by weights in descending order

Print out the documents whose relevant score is larger than 0